

Transmisión de Datos: de ICMP y ARP a Sockets TCP entre Dos Computadoras

- Objetivos:**
- Observar tráfico real con Wireshark e identificar la encapsulación Ethernet → IP → ICMP/TCP/UDP.
 - Comprender cómo un host descubre la dirección MAC de otro host de su LAN (ARP).
 - Experimentar con TCP y UDP usando ncat y comparar lo que se ve en la captura.
 - Implementar un cliente y un servidor con sockets (TCP y UDP) y relacionar el código con la red.
- Requisitos:**
- Dos computadoras con Wi-Fi (se trabaja en grupos de al menos 2) y un router domiciliario o teléfono que pueda compartir Internet (hotspot).
 - Wireshark
 - Python3
 - [Netcat](#)
 - Scripts tcp_server.py, tcp_client.py
- Conocimientos Previos:**
- Modelo en capas (OSI / TCP-IP) y direccionamiento IPv4 básico (IP, máscara, gateway)
 - Uso básico de la terminal (cmd / PowerShell en Windows, bash en Linux).
 - Python básico (variables, strings, ejecutar un script). No se asume experiencia con sockets.

Consignas:

Al final del TP tienen que poder contestar esa pregunta con detalle, y haber visto con Wireshark buena parte de la respuesta. Vamos a ir de lo más simple (un ping) a lo más propio (un programa escrito por ustedes), y de una sola computadora a dos.

Antes de empezar:

- Capturen solamente tráfico propio, en sus computadoras o en el router/hotspot del grupo. No analicen el tráfico de otras personas.
- Para capturar, Wireshark necesita permisos: en Windows alcanza con haber instalado Npcap; en Linux, ejecutar Wireshark con sudo o agregar su usuario al grupo wireshark (según la distribución).
- En Windows usen python (o py); en Linux, python3. En los ejemplos escribimos python.
- Guarden cada captura importante: forman parte de la entrega.
- Todos los filtros de este TP para wireshark son display filters (se escriben en la barra arriba de la lista de paquetes). No los confundan con los capture filters (que se configuran antes de capturar y usan otra sintaxis). Si el filtro está bien escrito la barra se pone verde; si está mal, roja.

Puertos que vamos a usar en todo el TP:

Protocolo	Puerto	Dónde se usa
TCP	12000	ncat (TCP), tcp_server.py, tcp_client.py, filtro tcp.port == 12000
UDP	12001	ncat -u (UDP), udp_server.py, udp_client.py, filtro udp.port == 12001

Usamos puertos distintos para que los filtros de TCP y de UDP no se mezclen al comparar capturas. (En el punto 4 van a pensar si realmente hacía falta que fueran distintos.)

Consignas:

1) ICMP y primer contacto con Wireshark.

Empezamos con la herramienta de red más conocida: ping. La vamos a usar como "generador de tráfico" para aprender a leer una captura. Investigar brevemente y documentar en el trabajo:

- ¿Qué es ICMP y para qué se usa? ¿Transporta datos de aplicaciones como lo hacen TCP o UDP?
- ¿Qué relación tiene con IP? ¿Viaja dentro de IP, al lado de IP o debajo de IP? ¿Cómo sabe el receptor que el contenido de un paquete IP es ICMP?
- ¿Qué hace ping? ¿Qué son un Echo Request y un Echo Reply? ¿Qué campos de ICMP permiten distinguirlos?
- ¿Qué información mínima contiene un mensaje ICMP de tipo Echo?

Averiguar la configuración de red de alguna de las computadoras del grupo: dirección IPv4, máscara, gateway por defecto y dirección MAC de la interfaz que usan (Wi-Fi o cableada).

```
Windows: ipconfig /all
Linux: ip addr y ip route
```

Abrir Wireshark, elegir la interfaz por la que salen a Internet (Wi-Fi o Ethernet; la que muestra actividad en el gráfico) y comenzar la captura. En una terminal, hacer ping a la IP del gateway por defecto y luego a una IP de Internet:

```
# Windows (envía 4 pings por defecto)
ping -4 <IP_del_gateway>
ping -4 8.8.8.8

# Linux (sin -c no se detiene; cortar con Ctrl+C)
```

```
ping -4 -c 4 <IP_del_gateway>
ping -4 -c 4 8.8.8.8
```

Usamos `-4` y direcciones IP (no nombres) para forzar IPv4. Si hacen ping a un nombre, puede usarse IPv6 y el tráfico aparece como `icmpv6`, que no coincide con el filtro `icmp`.

Detener la captura y aplicar el filtro:

```
icmp
```

Si la red no responde a pings hacia Internet, no es un error de ustedes: algunas redes filtran ICMP (aunque es raro, con datos del celu no debería pasarles). En ese caso trabajen con el ping al gateway.

Filtros útiles para esta parte (pruébenlos):

Filtro	Que muestra
<code>icmp</code>	todos los mensajes ICMP (IPv4)
<code>icmp.type == 8</code>	solo Echo Request
<code>icmp.type == 0</code>	solo Echo Reply
<code>icmp && ip.addr == 8.8.8.8</code>	el intercambio con esa IP (en ambos sentidos)

Seleccionar un Echo Request y desplegar el panel de detalles (el del medio). Identificar las capas que muestra Wireshark y completar una tabla como esta:

Capa (como la nombra Wireshark)	Dirección/identificador origen	Dirección/identificador destino	¿Qué campo indica qué protocolo viene "adentro"?
Ethernet II			
Internet Protocol Version 4			
Internet Control Message Protocol			
Datos / payload			

Ayuda: al hacer clic en una línea del panel de detalles, Wireshark resalta en el panel inferior (hexadecimal) los bytes que le corresponden. Úsenlo para "ver" dónde empieza y termina cada encabezado.

Responder, justificando con la captura (una captura de pantalla con el campo resaltado vale más que el valor copiado):

- La MAC destino del Echo Request enviado a 8.8.8.8, ¿es la MAC de 8.8.8.8? ¿De qué equipo es? Compárenla con la MAC destino del ping al gateway. ¿Qué conclusión sacan sobre el alcance de una dirección MAC frente al de una dirección IP?
- Comparen un Echo Request con su Echo Reply (Wireshark los vincula en el campo [Response frame: ...]). Hagan una lista de los campos que cambian y de los que se mantienen en Ethernet, IP e ICMP. ¿Por qué tiene sentido cada cambio? ¿Por qué el identificador y el número de secuencia se mantienen?
- ¿Dónde está el payload de ping? ¿Cuántos bytes tiene y qué contiene? ¿Es igual en el Reply? Si en el grupo hay una computadora con Windows y otra con Linux, compárenlos: ¿qué les sugiere que sean distintos?
- ¿Qué valor de TTL tiene el Echo Request que ustedes enviaron? ¿Y el Reply que llegó de 8.8.8.8? ¿Por qué no son iguales? (Pista: investiguen qué hace un router con el TTL.)
- Dibujen la encapsulación del paquete que eligieron como "cajas dentro de cajas", indicando para cada caja qué tamaño en bytes tiene según Wireshark.

2) ARP: de una IP a una dirección MAC

En el punto anterior la trama Ethernet ya tenía una MAC destino. Pero ping solo conoce una IP. ¿De dónde salió esa MAC? Investigar brevemente:

- ¿Qué problema resuelve ARP? ¿En qué capa lo ubicarían y por qué es discutible?
- ¿Qué es un ARP Request y un ARP Reply? ¿A quién se envía cada uno?
- ¿Qué es la caché ARP y por qué existe?
- Traten de responder con sus palabras: "Tengo la IP de una máquina de mi red local. ¿Cómo sé a qué dirección MAC debo enviarle la trama?"
- Ver la caché ARP de su computadora y buscar la entrada del gateway:

```
# Windows
arp -a

# Linux
ip neigh show
```

¿La MAC asociada al gateway coincide con la MAC destino que vieron anteriormente?

- f) Generar tráfico ARP y capturarlo. Capturen en la interfaz Wi-Fi/Ethernet con el filtro arp. Como las entradas quedan guardadas en la caché, un ping a un equipo con el que ya hablaron puede no generar ningún ARP. Tienen varias formas de provocarlo (elijan la que puedan usar):

Opción A (no requiere permisos de administrador): hacer ping a una IP de su misma subred que no esté en uso. Por ejemplo, si su IP es 192.168.1.37/24, prueben 192.168.1.253 (antes verifiquen con `arp -a` / `ip neigh` que no aparezca).

```
# Windows
ping -4 -n 2 192.168.1.253

# Linux
ping -4 -c 2 192.168.1.253
```

Opción B (requiere administrador; el cambio es temporal): borrar solo la entrada del gateway y volver a hacerle ping. La entrada se vuelve a aprender sola en cuanto se necesita; no rompe la configuración de red.

```
# Windows (cmd o PowerShell "Ejecutar como administrador")
arp -d <IP_del_gateway>
ping -4 -n 2 <IP_del_gateway>

# Linux (reemplazar wlan0 por el nombre de su interfaz, ver "ip addr")
sudo ip neigh del <IP_del_gateway> dev wlan0
ping -4 -c 2 <IP_del_gateway>
```

Opción C: hacer ping a un equipo de la LAN con el que no hayan hablado recientemente (por ejemplo, la computadora del compañero/a).

No usen comandos que modifiquen la configuración de la interfaz o agreguen entradas estáticas: no hace falta para este TP.

Filtros útiles!

Filtro	Que muestra
arp	todo ARP
arp.opcode == 1	solo ARP Request
arp.opcode == 2	solo ARP Reply

<code>arp.dst.proto_ipv4 == 192.168.1.1</code>	ARP que pregunta por esa IP (cambiar por la del gateway)
<code>arp icmp</code>	ARP e ICMP juntos
<code>eth.dst == ff:ff:ff:ff:ff:ff</code>	tramas enviadas a broadcast

Analizar un ARP Request y su ARP Reply. Para cada uno completar una tabla como la siguiente:

Campo	ARP Request	ARP Reply
MAC destino (encabezado Ethernet)		
MAC origen (encabezado Ethernet)		
Opcode		
Sender MAC address		
Sender IP address		
Target MAC address		
Target IP address		

Y responder:

- ¿Por qué el Request va a una dirección broadcast y el Reply no? ¿Qué valor tiene Target MAC address en el Request y por qué?
- En el encabezado Ethernet de la trama ARP, ¿qué valor tiene el campo Type? ¿Hay un encabezado IP? ¿Qué les dice eso sobre dónde "vive" ARP?
- Con la Opción A (IP inexistente): ¿cuántos ARP Request aparecieron? ¿Hubo Reply? ¿Apareció algún ICMP Echo Request en la captura? Expliquen por qué.
- Volvieron a hacer ping al mismo destino un minuto después: ¿apareció ARP de nuevo? Revisen la caché ARP. ¿Qué ventaja tiene la caché y qué problema podría causar si una entrada quedara vieja?

Nota: el comportamiento exacto de la caché (cuánto dura una entrada, si se revalida con un Request dirigido a una MAC concreta en vez de broadcast, etc.) depende del sistema operativo. Si ven algo distinto de lo que esperaban, descríbanlo: es parte de la observación.

3) TCP y UDP "a mano" con [ncat](#)

Hasta ahora observamos protocolos que usa el sistema operativo. Ahora vamos a comunicar dos procesos (dos programas) sin escribir código todavía. ncat es un programa que toma lo que escribimos en el teclado y lo manda por la red (y viceversa). La idea es empezar a generar tráfico cada vez más a bajo nivel (primero generamos tráfico con Packet Tracer, luego con la terminal del SO, ahora con un software a través de la CLI y finalmente ejecutaremos algunos scripts). Descargar netcat! para linux es bastante directo:

```
sudo apt install netcat-openbsd
```

Para windows, tienen la versión portable para descargar, es lo más cómodo pero seguramente tengan que apagar el defender un rato para bajarlo.

Investigar brevemente:

- a) ¿Qué significa "establecer una conexión"? ¿Dónde "existe" una conexión TCP: en los cables, en los routers o en los extremos?
- b) ¿Qué es un puerto? ¿Qué identifica el par (IP, puerto)?
- c) ¿Qué significa que un proceso esté "escuchando" en un puerto?

Importante: capturar en loopback. En esta parte cliente y servidor corren en la misma computadora y se comunican por 127.0.0.1 o localhost. Ese tráfico no sale por la placa Wi-Fi: en Wireshark hay que elegir la interfaz de loopback:

Windows (Npcap): Adapter for loopback traffic capture.

Linux: lo.

Van a notar que, según el sistema operativo, la captura en loopback no muestra un encabezado Ethernet o lo muestra con MAC 00:00:00:00:00:00.

Experimento TCP. Iniciar la captura en loopback con el filtro tcp.port == 12000. Abrir dos terminales:

```
# Terminal A (servidor): escucha en el puerto TCP 12000
ncat -v -l 12000
```

```
# Terminal B (cliente): se conecta al servidor
ncat -v 127.0.0.1 12000
```

Escriban una línea en la terminal B y presionen Enter; luego respondan desde la A. Manden 2 o 3 mensajes en cada sentido. Finalmente cierran el cliente (Ctrl+C en B) y detengan la captura. Capturen el paquete con Wireshark, recuerden filtrar con `tcp.port == 12000` así no se les hace una ensalada de paquetes en el sniffer.

Experimento UDP. Nueva captura en loopback, ahora con el filtro `udp.port == 12001`:

```
# Terminal A (servidor)
ncat -v -u -l 12001
```

```
# Terminal B (cliente)
ncat -v -u 127.0.0.1 12001
```

Envíen mensajes en ambos sentidos (el servidor puede responder una vez que recibió el primer datagrama del cliente) y cierren con Ctrl+C.

- ¿Qué pasó en la red cuando ejecutaron el comando del cliente, antes de escribir el primer mensaje? Compárenlo con TCP.
- ¿Cuántos datagramas generó cada mensaje? ¿Hay algo parecido a un ACK?
- Comparen el encabezado UDP con el encabezado TCP de un segmento con datos: ¿qué campos tiene cada uno? ¿Cuántos bytes ocupa cada encabezado?
- ¿Qué pasó en la red al cerrar el cliente con Ctrl+C? ¿Y en TCP?
- Para enviar la misma frase, ¿cuántos paquetes necesitaron en total con TCP y cuántos con UDP? ¿Qué "compran" con los paquetes extra de TCP?
- ¿Y si nadie escucha? Con Wireshark capturando en loopback y el filtro `tcp.port == 12000 || udp.port == 12001 || icmp`, sin servidores corriendo:

```
ncat -v 127.0.0.1 12000      # TCP a un puerto cerrado
ncat -v -u 127.0.0.1 12001  # UDP a un puerto cerrado, escribir y enter
```

4) Servidor TCP mínimo - TP5 Redes de Computadoras.

Un socket es el "punto final" de una comunicación que el sistema operativo le entrega a un programa: el programa escribe y lee bytes en el socket, y el sistema operativo se ocupa de TCP/UDP, IP y la placa de red. No vamos a entrar en tanta teoría aca, ya está bastante pesado el TP. A programar.

Ahora hacemos nosotros lo que hacía ncat. Los scripts están resueltos para que puedan concentrarse en relacionar cada línea con lo que ocurre en la red.

a) Leer el código del servidor:

```
import socket
import sys

HOST = sys.argv[1] if len(sys.argv) > 1 else "127.0.0.1"
PORT = int(sys.argv[2]) if len(sys.argv) > 2 else 12000

# 1. Crear el socket. AF_INET = IPv4, SOCK_STREAM = TCP.
#     Esto es una operación local: todavía no viaja nada por la red.
servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Permite reiniciar el servidor enseguida sin el error "Address already in use"
# (ver TIME_WAIT en la guía). No es imprescindible para entender el resto.
servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

# 2. Asociar el socket a una IP y un puerto locales (local, sin tráfico).
servidor.bind((HOST, PORT))

# 3. Pedirle al sistema operativo que acepte conexiones en ese puerto.
#     A partir de acá, un SYN que llegue a este puerto será respondido con SYN-ACK.
servidor.listen(1)
print(f"[servidor TCP] escuchando en {HOST}:{PORT} ...")

# 4. Esperar a que un cliente complete el three-way handshake.
#     accept() devuelve un socket NUEVO, dedicado a esta conexión,
#     y la dirección (IP, puerto) del cliente.
conexion, direccion_cliente = servidor.accept()
print(f"[servidor TCP] conexión aceptada desde {direccion_cliente[0]}:{direccion_cliente[1]}")

# 5. Recibir hasta 1024 bytes. TCP entrega un flujo de bytes: recv() puede
#     devolver menos (o más mensajes juntos) de lo que el cliente envió en un send().
datos = conexion.recv(1024)

# 6. Mostrar lo recibido (bytes -> texto).
mensaje = datos.decode("utf-8", errors="replace")
print(f"[servidor TCP] recibido ({len(datos)} bytes): {mensaje!r}")

# 7. Enviar una respuesta sencilla.
respuesta = f"Recibido: {mensaje}"
conexion.sendall(respuesta.encode("utf-8"))
print(f"[servidor TCP] respuesta enviada: {respuesta!r}")

# Esperar a que el cliente cierre: recv() devuelve b"" cuando llega su FIN.
while conexion.recv(1024):
```

```
pass
print("[servidor TCP] el cliente cerró la conexión")

conexion.close()
servidor.close()
```

Uso:

```
python tcp_server.py          # escucha en 127.0.0.1:12000 (solo esta computadora)
python tcp_server.py 0.0.0.0   # escucha en todas las interfaces (Parte 7)
python tcp_server.py 0.0.0.0 12000 # IP y puerto explícitos
```

b) Leer el código del cliente:

```
python tcp_client.py          # se conecta a 127.0.0.1:12000
python tcp_client.py 192.168.43.20 # IP del servidor (Parte 7)
python tcp_client.py 192.168.43.20 "Hola!" # IP y mensaje
```

Uso:

```
import socket
import sys

SERVER_IP = sys.argv[1] if len(sys.argv) > 1 else "127.0.0.1"
MENSAJE = sys.argv[2] if len(sys.argv) > 2 else "Hola servidor"
SERVER_PORT = 12000

# 1. Crear el socket TCP (operación local).
cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
cliente.settimeout(5) # evita quedar bloqueado para siempre si el servidor no responde

# 2. Conectarse: acá el sistema operativo envía el SYN y completa el
# three-way handshake. El puerto local del cliente lo elige el SO.
try:
    cliente.connect((SERVER_IP, SERVER_PORT))
except ConnectionRefusedError:
    sys.exit("[cliente TCP] conexión rechazada: el host respondió, pero nadie escucha en ese puerto (RST)")
except (socket.timeout, TimeoutError):
```

```
sys.exit("[cliente TCP] sin respuesta al intento de conexión (¿firewall? ¿IP
equivocada? ¿aislamiento de clientes?)" )

ip_local, puerto_local = cliente.getsockname()
print(f"[cliente TCP] conectado a {SERVER_IP}:{SERVER_PORT} desde
{ip_local}:{puerto_local}")

# 3. Enviar el mensaje (texto -> bytes).
cliente.sendall(MENSAJE.encode("utf-8"))
print(f"[cliente TCP] enviado: {MENSAJE!r}")

# 4. Recibir la respuesta.
datos = cliente.recv(1024)

# 5. Mostrarla.
print(f"[cliente TCP] respuesta ({len(datos)} bytes): {datos.decode('utf-8',
errors='replace')!r}")

# Cerrar: el SO envía un FIN y comienza el cierre de la conexión.
cliente.close()
```

c) Ejecutar (capturando en loopback con el filtro `tcp.port == 12000`):

```
# Terminal A
python tcp_server.py

# Terminal B
python tcp_client.py 127.0.0.1 "Hola desde mi programa"
```

Comportamiento esperado (el puerto del cliente va a ser otro):

```
[servidor TCP] escuchando en 127.0.0.1:12000 ...
[servidor TCP] conexión aceptada desde 127.0.0.1:48096
[servidor TCP] recibido (22 bytes): 'Hola desde mi programa'
[servidor TCP] respuesta enviada: 'Recibido: Hola desde mi programa'
[servidor TCP] el cliente cerró la conexión
```

```
[cliente TCP] conectado a 127.0.0.1:12000 desde 127.0.0.1:48096
[cliente TCP] enviado: 'Hola desde mi programa'
[cliente TCP] respuesta (32 bytes): 'Recibido: Hola desde mi programa'
```

El servidor atiende un cliente y termina. Para repetir, vuelvan a ejecutarlo.

- d) Con la captura, completar una tabla como la siguiente. Para cada llamada, indicar si genera tráfico y, si lo genera, qué segmentos (número de paquete en Wireshark y flags):

Llamada	¿Dónde se ejecuta?	¿Genera tráfico?	Segmentos que observan
socket()	servidor y cliente		
bind()	servidor		
listen()	servidor		
connect()	cliente		
accept()	servidor		
sendall()	servidor		
recv()	servidor		
close()	ambos		